BÀI TẬP LÝ THUYẾT
LẬP KẾ HOẠCH VÀ ĐỒ THỊ KẾ HOẠCH TRONG AI

Họ và tên	Hồ Hoàng Khang
MSSV / Lớp	3123410150

Mục tiêu học tập

• Mô tả bài toán lập kế hoạch cổ điển bằng trạng thái, mục tiêu, toán tử và các tập Precondition/Add/Delete. • Xây dựng planning graph theo GraphPlan với các mức trạng thái S_i, mức hành động A_i và các hành động No op.

• Nhận diện và giải thích các quan hệ mutex: Inconsistent Effects, Interference và Competing Needs. • Phân tích điều kiện dừng của GraphPlan và trích xuất kế hoạch từ planning graph.

• Vận dụng GraphPlan vào các bài toán Crane, Valet Parking và môi trường planning rời rạc. **Hướng dẫn**: Trả lời bằng tiếng Việt, trình bày rõ các bước suy luận. Với câu có planning graph, có thể vẽ trực tiếp lên hình hoặc bổ sung bảng. No-op được tính là một hành động khi xét mutex.

Bài 1. Khái niệm cơ bản trong GraphPlan

Hãy định nghĩa ngắn gọn các khái niệm sau và nêu ý nghĩa của chúng trong việc xây dựng planning graph. a) Interference (xung đột do can thiệp):

- **Định nghĩa:** Hai hành động a_1 và a_2 tại cùng một mức hành động A_i có quan hệ *Interference* nếu một trong hai hành động xóa (nằm trong danh sách Delete / tạo hiệu ứng phủ định) một tiền điều kiện (Precondition) của hành động còn lại.
- **Ý nghĩa:** Đảm bảo tính nhất quán về thứ tự thực thi: hai hành động này không thể thực hiện đồng thời hoặc theo bất kỳ thứ tự tuần tự nào trong cùng một bước thời gian, vì hành động này sẽ phá hủy điều kiện cần để hành động kia xảy ra.

b) Inconsistent Effects / Inconsistency (hiệu ứng không nhất quán):

- **Định nghĩa:** Hai hành động a_1 và a_2 tại cùng một mức A_i có quan hệ *Inconsistent Effects* nếu hiệu ứng (Add/Delete) của hành động này là phủ định của hiệu ứng từ hành động kia (tức là một hành động tạo ra literal p, còn hành động kia tạo ra $\neg p$ hoặc xóa p).
- **Ý nghĩa:** Ngăn chặn việc sinh ra các trạng thái mâu thuẫn logic (p và $\neg p$ cùng đúng tại một thời điểm) ở mức trạng thái kế tiếp S_{i+1}, đồng thời giúp xác định các cặp literal mutex ở S_{i+1}.

c) Competing Needs (tiền điều kiện cạnh tranh):

- **Định nghĩa:** Hai hành động a_1 và a_2 tại mức A_i có quan hệ *Competing Needs* nếu tồn tại một tiền điều kiện của a_1 và một tiền điều kiện của a_2 đang có quan hệ mutex với nhau ở mức trạng thái liền trước S_i.
- **Ý nghĩa:** Lan truyền các ràng buộc xung đột từ mức trạng thái S_i sang mức hành động A_i. Nó khẳng định rằng a_1 và a_2 không thể cùng được kích hoạt vì các điều kiện tiên quyết của chúng không thể đồng thời tồn tại ở trạng thái trước đó.

d) Có những cách nào để tìm một kế hoạch trong mô hình lập kế hoạch cổ điển?

Có 3 hướng tiếp cận chính để giải bài toán lập kế hoạch cổ điển:

- **Tìm kiếm trong không gian trạng thái (State-space search):**

    - *Tìm kiếm tiến (Forward / Progression search):* Bắt đầu từ trạng thái ban đầu S_0, áp dụng các hành động thỏa mãn tiền điều kiện để chuyển sang các trạng thái mới cho đến khi đạt trạng thái mục tiêu.
    - *Tìm kiếm lùi (Backward / Regression search):* Bắt đầu từ tập mục tiêu, chọn các hành động tạo ra các mục tiêu đó và lùi dần về các tiền điều kiện cho đến khi tập tiền điều kiện được thỏa mãn bởi trạng thái ban đầu.
- **Tìm kiếm trong không gian kế hoạch (Plan-space search / Partial-Order Planning - POP):** Bắt đầu từ một kế hoạch rỗng hoặc chưa hoàn chỉnh, sau đó tinh chỉnh dần bằng cách thêm hành động, thêm ràng buộc thứ tự và gán biến cho đến khi mọi tiền điều kiện đều được đáp ứng và không còn xung đột.
- **Chuyển đổi sang các bài toán thỏa mãn ràng buộc / suy diễn logic:**
    - Sử dụng **Planning Graph (GraphPlan)** để mở rộng đồ thị theo từng bước thời gian và trích xuất kế hoạch hợp lệ.
    - Mã hóa bài toán lập kế hoạch thành bài toán thỏa mãn tính đúng sai của mệnh đề logic (**SATPlan**) hoặc bài toán thỏa mãn ràng buộc (**CSP**).

e) Khi dùng heuristic dựa trên GraphPlan, những giả định nào của classical planning được nới lỏng? Vì sao điều này hữu ích hơn so với tìm kiếm ngây thơ?

Các giả định được nới lỏng (Relaxation):

1. *Bỏ qua danh sách xóa (Ignore Delete Lists):* Giả định các hành động không bao giờ làm mất đi các điều kiện đã đạt được (mọi literal đã xuất hiện đều tồn tại mãi mãi).
2. *Cho phép thực hiện song song và bỏ qua tương tác tiêu cực (bỏ qua mutex):* Khi tính giá trị heuristic (như max-level, level-sum hoặc set-level), hệ thống thường bỏ qua xung đột giữa các hành động, coi như các hành động có thể chạy đồng thời mà không cản trở lẫn nhau.

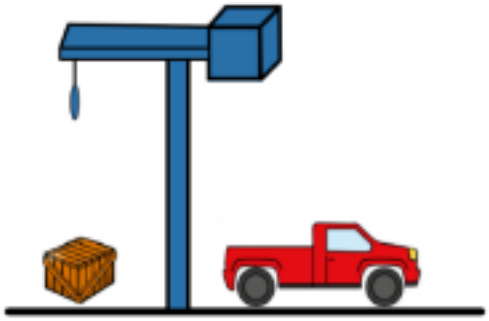
Lý do hữu ích hơn tìm kiếm ngây thơ (Uninformed Search):

- **Giảm mạnh không gian tìm kiếm và hệ số nhánh:** Tìm kiếm ngây thơ phải duyệt mù quáng qua toàn bộ tổ hợp trạng thái dẫn đến bùng nổ tổ hợp; heuristic từ GraphPlan cung cấp ước lượng khoảng cách sát thực tế từ trạng thái hiện tại đến mục tiêu, định hướng thuật toán tìm kiếm (như A^*, Greedy Best-First Search) đi thẳng vào nhánh triển vọng.
- **Tính toán nhanh và có tính chất chấp nhận được (Admissible):** Việc xây dựng relaxed planning graph chỉ mất thời gian đa thức (do không xét xóa và mutex). Chi phí ước lượng (đặc biệt là *max-level* và *set-level*) không bao giờ đánh giá cao hơn chi phí thực tế, giúp đảm bảo tìm được lời giải tối ưu nhanh chóng hơn nhiều so với tìm kiếm không thông tin.

Bài 2. Lập kế hoạch ký hiệu - Bài toán cần cẩu

Cho một cần cẩu, một kiện hàng và một xe tải. Ban đầu kiện hàng ở bên trái, xe tải ở bên phải và cần cẩu quay về bên trái. Mục tiêu là đưa kiện hàng lên xe tải và cuối cùng cần cẩu quay về bên trái.

Cần cẩu có thể quay giữa trái và phải, có hoặc không mang kiện hàng. Cần cẩu có thể nhấc kiện hàng nếu cả hai ở cùng phía. Kiện hàng chỉ có thể được đưa lên xe tải bằng cần cẩu.



a) Xác định các biến/đối tượng, các predicate cần dùng, trạng thái ban đầu và trạng thái mục tiêu.

Các đối tượng (Objects) / Hằng số:

- **Vị trí:** Left, Right (các biến vị trí $x, y \in \{Left, Right\}$, với $x \neq y$).
- **Đối tượng thực thể:** Crane (cần cẩu), Crate (kiện hàng), Truck (xe tải) — có thể ẩn vào tên predicate vì mỗi loại chỉ có 1 thực thể duy nhất.

Các vị từ (Predicates):

- **CraneAt(x):** Cần cẩu đang quay về phía x ($x \in \{Left, Right\}$).
- **CrateAt(x):** Kiện hàng đang đặt trên mặt đất ở phía x ($x \in \{Left, Right\}$).
- **Holding:** Cần cẩu đang nhấc/giữ kiện hàng.
- **Empty:** Cần cẩu đang rỗng (không giữ kiện hàng).
- **InTruck:** Kiện hàng đã được đặt lên xe tải (biết rằng xe tải cố định ở Right nên kiện hàng chỉ lên được xe tải khi ở Right).

Trạng thái ban đầu (S_0):

$$CraneAt(Left) \wedge CrateAt(Left) \wedge Empty$$

(Có thể bổ sung các literal phủ định theo giả định thế giới đóng nếu cần biểu diễn tường minh: $\neg\text{CraneAt}(\text{Right}) \wedge \neg\text{CrateAt}(\text{Right}) \wedge \neg\text{Holding} \wedge \neg\text{InTruck}$*).*

Trạng thái mục tiêu ($Goal$):

$$InTruck \wedge CraneAt(Left)$$

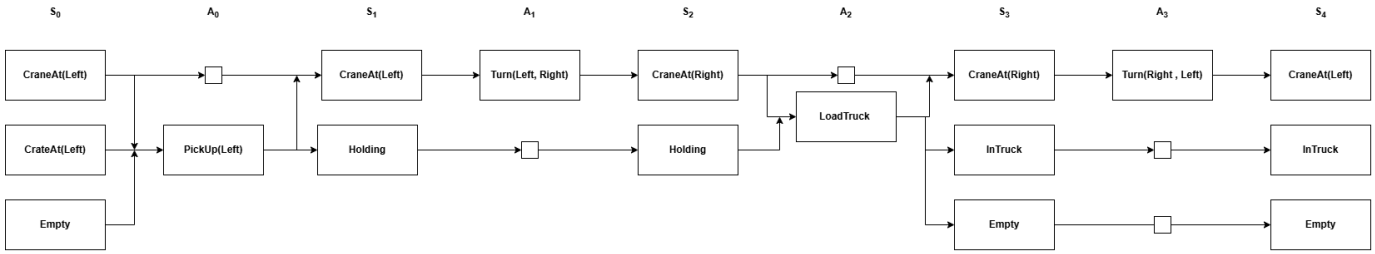
b) Xây dựng các toán tử hành động. Với mỗi toán tử, ghi rõ Preconditions, Add List và Delete List.

Có 4 nhóm toán tử cơ bản mô tả đầy đủ hoạt động của cần cẩu:

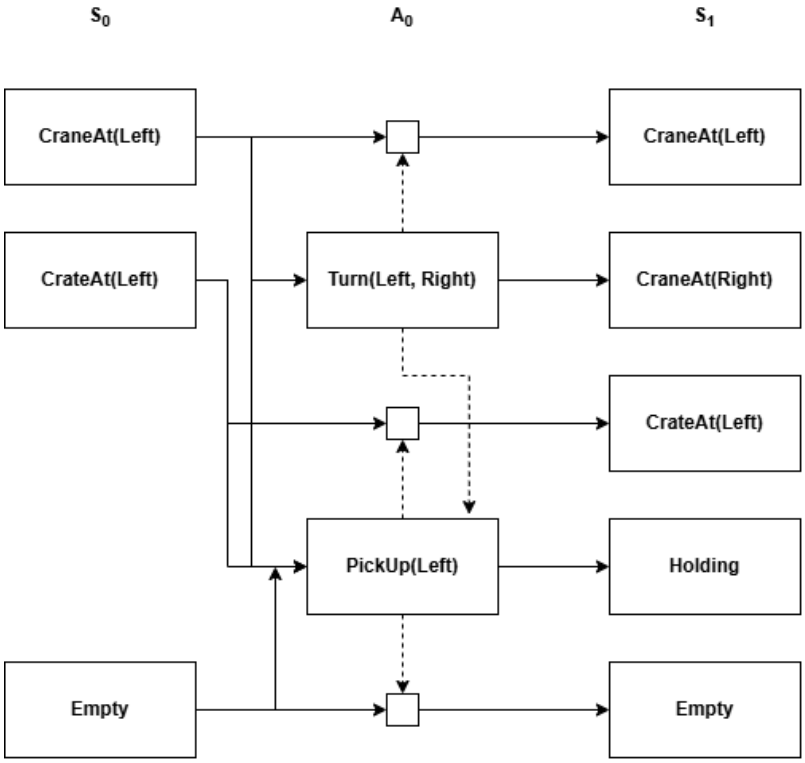
1. **Turn(x, y)** (Cần cẩu quay từ phía x sang phía y, với $x \neq y$ — áp dụng được cả khi đang giữ hàng hoặc đang rỗng):
   - **Preconditions:** $CraneAt(x)$
   - **Add List:** $CraneAt(y)$
   - **Delete List:** $CraneAt(x)$
2. **PickUp(x)** (Cần cẩu nhấc kiện hàng ở phía x):
   - **Preconditions:** $CraneAt(x), CrateAt(x), Empty$
   - **Add List:** $Holding$
   - **Delete List:** $CrateAt(x), Empty$
3. **PutDown(x)** (Cần cẩu hạ kiện hàng xuống mặt đất ở phía x):
   - **Preconditions:** $CraneAt(x), Holding$
   - **Add List:** $CrateAt(x), Empty$
   - **Delete List:** $Holding$

4. **LoadTruck** (Cần cẩu đặt kiện hàng lên xe tải ở phía phải):
    - **Preconditions:** $CraneAt(Right), Holding$
    - **Add List:** $InTruck, Empty$
    - **Delete List:** $Holding$

c) Vẽ planning graph cho 3 bước đầu tiên. Có thể dùng hình minh họa thay cho proposition nếu biểu diễn rõ ràng.



d) Vẽ hai mức đầu tiên của GraphPlan và chỉ ra các cặp hành động mutex; với mỗi cặp, ghi loại mutex.



Tại mức S_0 không có cặp literal nào xung đột nên ở mức A_0 không phát sinh quan hệ mutex loại *Competing Needs*. Các cặp hành động mutex tại mức A_0 bao gồm:

1. **Turn(Left, Right) và No-op(CraneAt(Left)):**
    - **Loại mutex: Inconsistent Effects và Interference.**
    - **Giải thích: Hành động Turn(Left, Right) xóa CraneAt(Left), mâu thuẫn với hiệu ứng duy trì CraneAt(Left) của No-op (Inconsistent Effects), đồng thời xóa mất tiền điều kiện cần thiết của No-op này (Interference).**
2. **Turn(Left, Right) và PickUp(Left):**
    - **Loại mutex: Interference.**
    - **Giải thích: Hành động Turn(Left, Right) xóa CraneAt(Left), trong khi CraneAt(Left) là tiền điều kiện (Precondition) bắt buộc để thực hiện PickUp(Left).**
3. **PickUp(Left) và No-op(CrateAt(Left)):**
    - **Loại mutex: Inconsistent Effects và Interference.**

- **Giải thích: Hành động PickUp(Left) xóa CrateAt(Left), đối nghịch với hiệu ứng giữ nguyên CrateAt(Left) của No-op và làm mất tiền điều kiện của No-op này.**

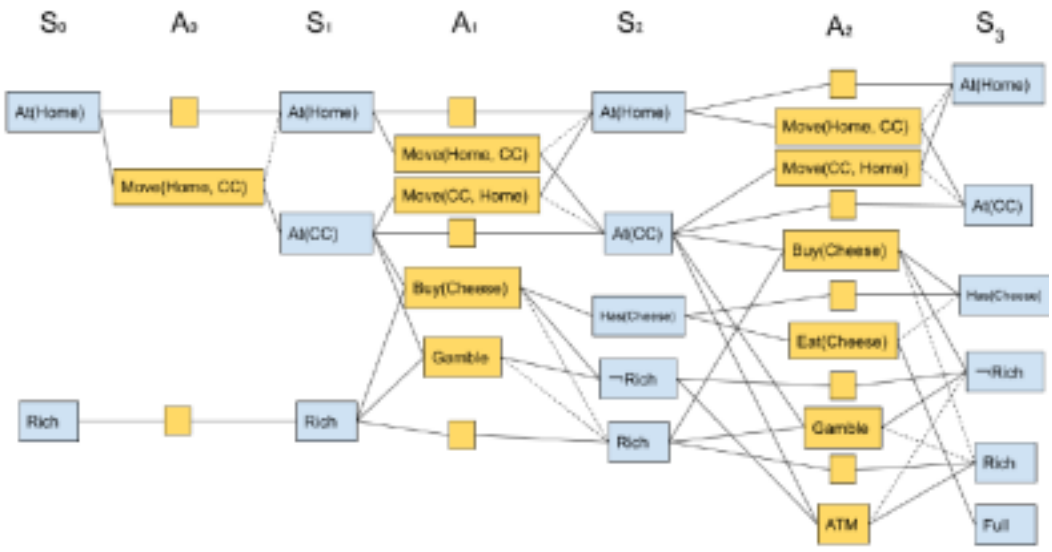
4. **PickUp(Left) và No-op(Empty):**
   - **Loại mutex: Inconsistent Effects và Interference.**
   - **Giải thích: Hành động PickUp(Left) xóa Empty, đối nghịch với hiệu ứng giữ nguyên Empty của No-op và làm mất tiền điều kiện của No-op này.**

Bài 3. Quan hệ Mutex trong planning graph

Pinky đi lấy thức ăn. Các hành động được cho như sau:

Hành động	Preconditions	Add List	Delete List
Move(A,B)	At(A)	At(B)	At(A)
Buy(Cheese)	At(ChuckyCheese), Rich	Has(Cheese), ¬Rich	
Gamble	At(ChuckyCheese), Rich	¬Rich	Rich
ATM	At(ChuckyCheese), ¬Rich	Rich	¬Rich
Eat(Cheese)	Has(Cheese)	Full	Has(Cheese)

Trạng thái bắt đầu: Rich ∧ At(Home). Mục tiêu: Full. Planning graph tương ứng được cho trong hình dưới:
Lập kế hoạch và đồ thị kế hoạch trong AI 3 / 6



Planning graph cho bài toán Pinky.

a) Ở mức A_0, hãy chỉ ra một cặp hành động mutex do Inconsistent Effects.

Chọn cặp **Move(Home, CC)** và **No-op(At(Home))**.

Nhìn trên hình: Đường nét đứt (Delete) từ Move(Home, CC) chĩa vào At(Home) ở S_1, xung đột trực tiếp với đường liền (Add) từ ô vuông No-op(At(Home)) nối sang At(Home) ở S_1.

b) Ở mức A_1, hãy chỉ ra một cặp hành động mutex do Interference.

Chọn cặp **Buy(Cheese)** và **No-op(Rich)** (hoặc **Gamble** và **No-op(Rich)**).

Nhìn trên hình: Đường nét đứt (Delete) từ Buy(Cheese) (và từ Gamble) trỏ xuống Rich ở S_2, tức là xóa mất tiền điều kiện Rich mà No-op(Rich) ở A_1 cần để thực hiện.

c) Ở mức A_2, hãy chỉ ra một cặp hành động mutex do Competing Needs.

Chọn cặp **Buy(Cheese)** và **ATM** (hoặc **Gamble** và **ATM**).
Nhìn trên hình: Buy(Cheese) nối từ Rich ở S_2, còn ATM nối từ ¬Rich ở S_2. Vì Rich và ¬Rich ở mức S_2 là hai mệnh đề đối nghịch (mutex với nhau), nên hai hành động này bị mutex theo loại **Competing Needs**.

d) Giải thích ngắn gọn vì sao mỗi cặp ở trên thỏa loại mutex tương ứng.

Cặp ở câu a (Move(Home, CC) và No-op(At(Home)) ở mức A_0 — *Inconsistent Effects*):

- Hành động Move(Home, CC) xóa At(Home) (Delete List chứa At(Home)), trong khi hành động No-op(At(Home)) giữ lại At(Home) ở mức tiếp theo (Add List chứa At(Home)). Hiệu ứng của hai hành động này trực tiếp phủ định lẫn nhau.

Cặp ở câu b (Buy(Cheese) và No-op(Rich) ở mức A_1 — *Interference*):

- Hành động Buy(Cheese) tạo ra ¬Rich và xóa mất Rich, làm phá hủy tiền điều kiện (Precondition) Rich mà hành động No-op(Rich) cần để thực hiện.

Cặp ở câu c (Buy(Cheese) và ATM ở mức A_2 — *Competing Needs*):

- Hành động Buy(Cheese) yêu cầu tiền điều kiện là Rich, còn hành động ATM yêu cầu tiền điều kiện là ¬Rich. Tại mức trạng thái S_2 trước đó, hai mệnh đề Rich và ¬Rich đối nghịch nhau nên chúng có quan hệ mutex. Vì có hai tiền điều kiện mutex với nhau ở mức trước nên Buy(Cheese) và ATM không thể thực hiện đồng thời.

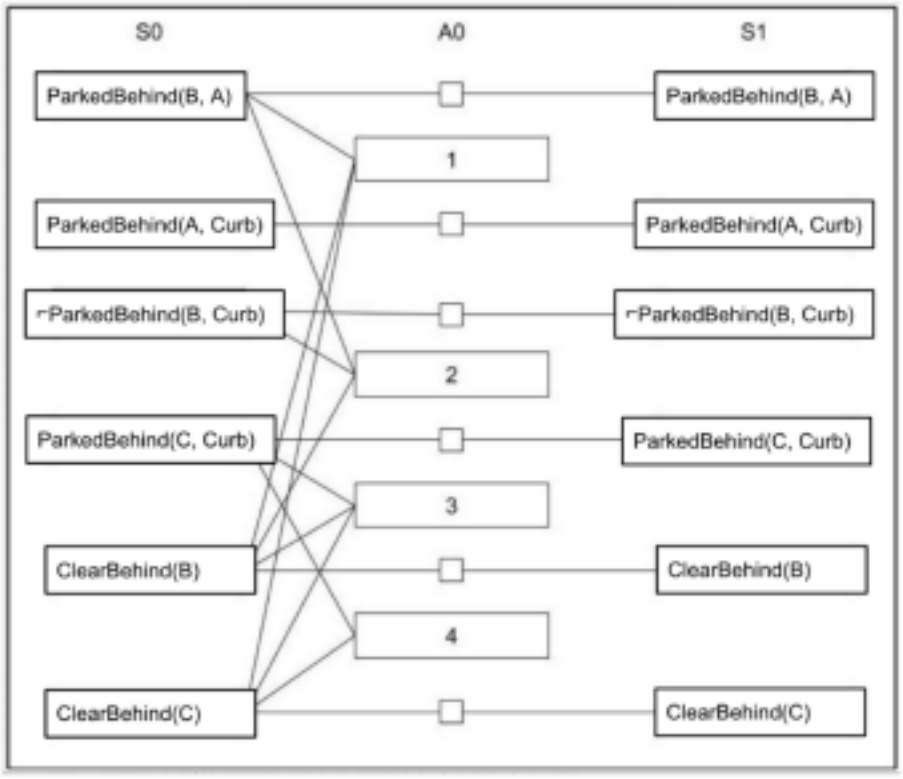
Bài 4. Classical Planning và GraphPlan - Valet Parking

Xét bài toán Valet Parking với hai predicate ClearBehind(car) và ParkedBehind(car1, car2). Hai toán tử được định nghĩa như sau:

Thành phần	**ParkBehind(car1, car2)**	**ParkInNewRow(car)**
Preconditions	ClearBehind(car1) ClearBehind(car2) ParkedBehind(car1, place)	ClearBehind(car) ParkedBehind(car, place) ¬ParkedBehind(car, curb)
Add List	ParkedBehind(car1, car2) ClearBehind(place) ¬ParkedBehind(car1, place) ¬ClearBehind(car2)	ParkedBehind(car, curb) ClearBehind(place) ¬ParkedBehind(car, place)
Delete List	ClearBehind(car2) ParkedBehind(car1, place)	ParkedBehind(car, place) ¬ParkedBehind(car, curb)

Lập kế hoạch và đồ thị kế hoạch trong AI 4 / 6

Hình dưới là mẫu cho hai mức đầu tiên $S_0 \rightarrow A_0 \rightarrow S_1$. Các nút hành động No-op đã được biểu diễn bằng các ô vuông nhỏ không tên.



Mẫu GraphPlan cho Valet Parking

a) Điền tên hành động phù hợp vào các ô 1-4 ở mức A_0. Nếu không tồn tại hành động hợp lệ cho tập tiền điều kiện đó, ghi "N/A".

Dựa vào các đường nối tiền điều kiện (Preconditions) từ cột S_0 vào từng ô 1, 2, 3, 4 ở cột A_0:

- **Ô 1: ParkBehind(B, C)**
  - *Phân tích đường nối:* Nhận 3 tiền điều kiện từ S_0 gồm ParkedBehind(B, A) (ứng với ParkedBehind(car1, place)), ClearBehind(B) (ClearBehind(car1)), và ClearBehind(C) (ClearBehind(car2)). Tất cả đều khớp hoàn toàn với định nghĩa của ParkBehind(B, C).
- **Ô 2: ParkInNewRow(B)**
  - *Phân tích đường nối:* Nhận 3 tiền điều kiện từ S_0 gồm ParkedBehind(B, A) (ứng với ParkedBehind(car, place)), ¬ParkedBehind(B, Curb) (¬ParkedBehind(car, curb)), và ClearBehind(B) (ClearBehind(car)). Khớp hoàn toàn với định nghĩa của ParkInNewRow(B).
- **Ô 3: ParkBehind(C, B)**
  - *Phân tích đường nối:* Nhận 3 tiền điều kiện từ S_0 gồm ParkedBehind(C, Curb) (ứng với ParkedBehind(car1, place)), ClearBehind(B) (ClearBehind(car2)), và ClearBehind(C) (ClearBehind(car1)). Khớp hoàn toàn với định nghĩa của ParkBehind(C, B).
- **Ô 4: N/A**
  - *Phân tích đường nối:* Nhận 2 đường nối từ ParkedBehind(C, Curb) và ClearBehind(C). Không có toán tử nào chỉ dùng 2 tiền điều kiện này (nếu muốn dùng ParkInNewRow(C) thì cần thêm ¬ParkedBehind(C, Curb), nhưng thực tế xe C đã đỗ ở Curb rồi). Do đó không tồn tại hành động hợp lệ.

b) Với từng hành động 1-4, liệt kê TẤT CẢ predicate ở S_1 nhận add-edge từ hành động đó. Nếu hành động không hợp lệ, ghi "N/A".

Hành động 1 — ParkBehind(B, C) (với car1 = B, car2 = C, place = A):

- ParkedBehind(B, C)
- ClearBehind(A)
- ¬ParkedBehind(B, A)
- ¬ClearBehind(C)

Hành động 2 — ParkInNewRow(B) (với car = B, place = A):

- ParkedBehind(B, Curb)
- ClearBehind(A)
- ¬ParkedBehind(B, A)

Hành động 3 — ParkBehind(C, B) (với car1 = C, car2 = B, place = Curb):

- ParkedBehind(C, B)
- ClearBehind(Curb)
- ¬ParkedBehind(C, Curb)
- ¬ClearBehind(B)

Hành động 4: N/A

c) Nêu hai action node có quan hệ mutex do Inconsistent Effects. No-op được phép sử dụng.

Cặp hành động: ParkBehind(B, C) (ô 1) và **No-op(ParkedBehind(B, A))**.

- (Hoặc bạn cũng có thể chọn: ParkBehind(B, C) và No-op(ClearBehind(C)); hoặc ParkInNewRow(B) và No-op(¬ParkedBehind(B, Curb)); hoặc ParkBehind(C, B) và No-op(ClearBehind(B))).

Giải thích: ParkBehind(B, C) tạo ra ¬ParkedBehind(B, A) (và xóa ParkedBehind(B, A)), mâu thuẫn trực tiếp với hiệu ứng duy trì ParkedBehind(B, A) của No-op(ParkedBehind(B, A)).

d) Nêu hai action node có quan hệ mutex do Interference.

Cặp hành động: ParkBehind(B, C) (ô 1) và **ParkBehind(C, B)** (ô 3).

- (Hoặc bạn có thể chọn: ParkBehind(B, C) và ParkInNewRow(B), vì cả hai cùng xóa ParkedBehind(B, A) là tiền điều kiện của nhau).

Giải thích: ParkBehind(B, C) xóa ClearBehind(C), trong khi ClearBehind(C) là tiền điều kiện (Precondition) bắt buộc của ParkBehind(C, B) (và ngược lại, ParkBehind(C, B) xóa ClearBehind(B) là tiền điều kiện của ParkBehind(B, C)).

e) GraphPlan "leveled off" khi nào? Chọn một đáp án:

☐ A. Tất cả các hành động có thể đã được khám phá.

☐ B. Không còn tập literal khác rỗng nào không có liên kết mutex.

☑ C. Hai mức liên tiếp của planning graph giống hệt nhau.

Lập kế hoạch và đồ thị kế hoạch trong AI 5 / 6

☐ D. Mức trạng thái cuối cùng chứa một trạng thái mục tiêu.

Bài 5. Xây dựng GraphPlan và trích xuất kế hoạch

Cho môi trường lập kế hoạch với 6 toán tử. Trạng thái bắt đầu là A; mục tiêu là C ∧ D ∧ E. Mỗi thời điểm chỉ áp dụng một toán tử và cần tìm kế hoạch có ít thao tác nhất.

	op1	**op2**	**op3**	**op4**	**op5**	**op6**
Precondition	A	B	A	A	A	A
Add	B	C, D, E	C	D	E	E, ¬A

Delete						

a) Chạy GraphPlan: vẽ lần lượt các mức trạng thái và mức hành động cho đến khi thuật toán dừng. Nhớ bổ sung các No-op cho trạng thái cần duy trì.



Khung để hoàn thiện planning graph; trạng thái đầu $S_0 = \{A\}$.

*(Lưu ý quy tắc bài toán: Trạng thái đầu $S_0 = \{A\}$, mục tiêu $Goal = C \wedge D \wedge E$, và đề bài quy định rõ **"Mỗi thời điểm chỉ áp dụng một toán tử"** — tức là ở mỗi mức hành động A_i, các hành động thực tế đều đôi một mutex với nhau nên không thể thực hiện song song hai hành động thực tế cùng lúc)*.

- **Mức S_0:**
  - A
- **Mức A_0** (tất cả các toán tử có Precondition = A kèm theo hành động duy trì No-op(A)):
  - No-op(A)
  - op1 (Pre: A → Add: B)
  - op3 (Pre: A → Add: C)
  - op4 (Pre: A → Add: D)
  - op5 (Pre: A → Add: E)
  - op6 (Pre: A → Add: E, ¬A)
- **Mức S_1** (tập hợp các mệnh đề tạo ra từ A_0):
  - A, B, C, D, E, ¬A
  - (Phân tích tại S_1: Mặc dù cả 3 mục tiêu C, D, E đều đã xuất hiện ở S_1, nhưng vì ở A_0 mỗi bước chỉ được áp dụng **một** toán tử nên op3 (tạo C), op4 (tạo D), và op5/op6 (tạo E) đôi một mutex với nhau. Do đó tại S_1, các cặp {C, D}, {C, E}, {D, E} đều **bị mutex** → thuật toán chưa thể dừng ở S_1 mà phải mở rộng tiếp sang $A_1 \rightarrow S_2$).
- **Mức A_1** (các toán tử có đủ Precondition ở S_1 kèm theo No-op cho 6 mệnh đề ở S_1):
  - 6 hành động duy trì: No-op(A), No-op(B), No-op(C), No-op(D), No-op(E), No-op(¬A)
  - Các toán tử thực tế: op1, op2 (do S_1 đã có B), op3, op4, op5, op6

- **Mức S_2:**
  - A, B, C, D, E, ¬A
  - *(Tại S_2: Toán tử op2 ở A_1 đồng thời tạo ra cả 3 mệnh đề C, D, E trong một bước duy nhất nên tại S_2, ba mục tiêu C, D, E đã xuất hiện đầy đủ và **không còn mutex với nhau** → **Thuật toán GraphPlan dừng tại** S_2).*

*(Khi vẽ vào khung trống trong file Word hoặc Draw.io, bạn vẽ 5 cột từ trái qua phải: **S0** → **A0** → **S1** → **A1** → **S2**, vẽ các ô chữ nhật cho trạng thái/toán tử và ô vuông nhỏ cho các No-op tương tự mẫu của Bài 3 và Bài 4).*

b) Kế hoạch nào được GraphPlan trả về?

Kế hoạch trả về gồm 2 bước (2 thao tác thực tế):

1. **Bước 1 (tại A_0):** Thực hiện **op1** (từ A tạo ra B).
2. **Bước 2 (tại A_1):** Thực hiện **op2** (từ B tạo ra đồng thời C, D, E).

Viết gọn: **[op1, op2]** (hoặc op1 -> op2).

c) Kế hoạch trên có tối ưu về số lượng thao tác hay không? ☑ Có ☐ Không

d) Liệt kê TẤT CẢ cặp toán tử mutex ở A_0 và TẤT CẢ cặp proposition mutex ở S_1. Nếu không có, ghi "Không có".

Tất cả các cặp toán tử mutex ở A_0:

1. **Do ràng buộc bài toán (*"Mỗi thời điểm chỉ áp dụng một toán tử"*):** Tất cả 10 cặp ghép đôi giữa 5 toán tử thực tế {op1, op3, op4, op5, op6} đều đôi một mutex với nhau:
   - (op1, op3), (op1, op4), (op1, op5), (op1, op6)
   - (op3, op4), (op3, op5), (op3, op6)
   - (op4, op5), (op4, op6)
   - (op5, op6)
2. **Do xung đột hiệu ứng (*Inconsistent Effects*):**
   - (op6, No-op(A)) — vì op6 tạo ra ¬A, đối nghịch trực tiếp với No-op(A) duy trì A. *(Lưu ý: Các cặp giữa op1, op3, op4, op5 với No-op(A) **không mutex** vì không toán tử nào trong số này xóa A).*

Tất cả các cặp proposition (mệnh đề) mutex ở S_1:

1. **Mệnh đề đối nghịch trực tiếp:**
   - (A, ¬A)
2. **Do chỉ được sinh ra bởi các toán tử mutex ở A_0 (*Inconsistent Support* — mọi cặp hành động tạo ra hai mệnh đề này đều mutex với nhau):**
   - (B, C), (B, D), (B, E), (B, ¬A)
   - (C, D), (C, E), (C, ¬A)
   - (D, E), (D, ¬A)
   - *(Lưu ý: E và ¬A **không mutex** ở S_1 vì cùng được tạo ra bởi một toán tử duy nhất là op6; đồng thời A cũng **không mutex** với B, C, D, E vì No-op(A) có thể chạy đồng thời với op1, op3, op4, op5).*

Bài 6. Tổng hợp và phân tích

a) So sánh tìm kiếm trực tiếp trong không gian trạng thái với GraphPlan. Trong trường hợp nào planning graph giúp giảm đáng kể không gian tìm kiếm?

So sánh:

- **Tìm kiếm trực tiếp trong không gian trạng thái (State-space search):** Duyệt qua từng trạng thái cụ thể một cách tuần tự (tiến từ trạng thái đầu hoặc lùi từ mục tiêu). Khi số lượng đối tượng và hành động lớn, hệ số phân nhánh (branching factor) bùng nổ theo cấp số nhân, dễ sa lầy vào các nhánh ngõ cụt.
- **GraphPlan:** Xây dựng một cấu trúc đồ thị phân tầng (Planning Graph) lưu đồng thời tất cả các mệnh đề và hành động khả dĩ theo từng bước thời gian, kèm theo các ràng buộc loại trừ lẫn nhau (**mutex**) để cắt tỉa sớm các tổ hợp bất khả thi trước khi trích xuất lời giải. Đồ thị được xây dựng trong thời gian đa thức.

Trường hợp planning graph giúp giảm đáng kể không gian tìm kiếm:

- Khi bài toán có **nhiều mục tiêu con độc lập hoặc có thể thực hiện song song**, có **nhiều hành động gây nhiễu/không liên quan**, hoặc khi **các ràng buộc xung đột (mutex) giúp loại bỏ sớm các nhánh cụt** ngay từ các tầng đầu (giúp ước lượng chính xác chi phí tối thiểu và tránh quay lui vô ích trong không gian trạng thái khổng lồ).

b) Vì sao No-op quan trọng trong GraphPlan? Nếu bỏ toàn bộ No-op thì việc biểu diễn trạng thái tồn tại qua nhiều mức sẽ bị ảnh hưởng như thế nào?

Vai trò của No-op (No-operation / Persistence action):

- No-op đóng vai trò là cầu nối chuyển tiếp một mệnh đề (literal) từ mức trạng thái S_i sang mức S_{i+1} khi không có hành động thực tế nào làm thay đổi nó.
- Nhờ có No-op, GraphPlan giữ được tính đồng nhất (mọi mệnh đề tồn tại ở S_{i+1} đều phải là kết quả của ít nhất một hành động ở A_i) và dễ dàng phát hiện xung đột (**mutex**) giữa việc *giữ nguyên một trạng thái* với một hành động thực tế *xóa bỏ trạng thái đó*.

Hậu quả nếu bỏ toàn bộ No-op:

- Một mệnh đề nếu không được một hành động thực tế nào tạo lại ở bước tiếp theo sẽ **tự động biến mất** khỏi các mức trạng thái sau đó ($S_{i+1}, S_{i+2}, \dots$), vi phạm nguyên lý quán tính (trạng thái vẫn giữ nguyên nếu không bị tác động).
- Đồ thị sẽ mất đi các xung đột *Inconsistent Effects* và *Interference* giữa hành động thực tế với trạng thái đang duy trì, dẫn đến việc **không thể trích xuất được kế hoạch hợp lệ** (vì các điều kiện đạt được từ bước trước không thể bảo toàn để phối hợp với các điều kiện đạt được ở bước sau).

c) Cho một mục tiêu gồm nhiều literal. Hãy giải thích vai trò của mutex giữa các literal mục tiêu trong quyết định mở rộng planning graph hoặc kết luận thất bại.

Quyết định mở rộng planning graph:

- Điều kiện cần để tồn tại kế hoạch ở mức S_i là **tất cả** literal của mục tiêu phải xuất hiện ở S_i **và không có bất kỳ cặp literal mục tiêu nào mutex với nhau**.
- Nếu tất cả literal mục tiêu đã xuất hiện ở S_i nhưng vẫn còn ít nhất một cặp bị **mutex**, điều đó chứng tỏ chúng chưa thể đồng thời thỏa mãn tại thời điểm i (do xung đột hành động hoặc thiếu bước trung gian). Thuật toán **bắt buộc phải mở rộng tiếp** sang mức $A_i \rightarrow S_{i+1}$.

Quyết định kết luận thất bại (Failure):

- Nếu planning graph đã đạt trạng thái bão hòa (**leveled off** — hai mức liên tiếp giống hệt nhau cả về tập nút lẫn tập mutex) mà các literal mục tiêu vẫn chưa xuất hiện đủ **hoặc vẫn còn cặp literal mục tiêu mutex với nhau**, thuật toán kết luận **bài toán vô nghiệm (thất bại)** và dừng ngay mà không rơi vào vòng lặp vô hạn.

d) Nêu quy trình tổng quát để trích xuất một kế hoạch từ planning graph sau khi tất cả literal mục tiêu đã xuất hiện ở cùng một mức và không mutex với nhau.

Sau khi đạt mức S_n chứa đầy đủ tập mục tiêu G không mutex, GraphPlan thực hiện tìm kiếm quay lui (Backward Search) từ mức cuối S_n lùi về S_0 theo 4 bước:

1. **Khởi tạo tại mức S_n: Đặt tập mục tiêu cần thỏa mãn tại mức $i = n$ là $G_n = G$.**
2. **Chọn tập hành động hỗ trợ tại mức A_{i-1}:**
   - **Với mỗi literal $g \in G_i$, chọn một hành động $a \in A_{i-1}$ (có thể là hành động thực tế hoặc No-op) sao cho $g \in Add(a)$.**
   - **Tập hợp các hành động được chọn Act_{i-1} phải thỏa điều kiện không có hai hành động nào trong Act_{i-1} mutex với nhau.**
3. **Lùi về mức trạng thái trước đó (S_{i-1}):**
   - **Xác định tập mục tiêu con mới cần đạt ở mức S_{i-1} bằng hợp tất cả tiền điều kiện của các hành động vừa chọn:**

     $$G_{i-1} = \bigcup_{a \in Act_{i-1}} Preconditions(a)$$

   - **Lặp lại bước 2 và 3 cho đến khi lùi về tới S_0 ($i = 0$), khi đó dãy $[Act_0, Act_1, \dots, Act_{n-1}]$ chính là kế hoạch lời giải (bỏ qua các No-op khi xuất kết quả cuối).**
4. **Quay lui (Backtracking) và mở rộng thêm nếu thất bại:**
   - **Nếu tại mức A_{i-1} không thể chọn được tập hành động Act_{i-1} không mutex để thỏa mãn G_i, thuật toán quay lui để thử tổ hợp hành động khác.**
   - **Nếu đã thử mọi tổ hợp mà vẫn không trích xuất được kế hoạch ở mức S_n (lưu vào bảng ghi nhớ *nogood* để tránh lặp lại), thuật toán sẽ mở rộng planning graph thêm một mức ($A_n \rightarrow S_{n+1}$) rồi lặp lại quá trình trích xuất từ S_{n+1}.**

— Hết —